Problem Set 7

> In the completion of Problem 3, Problem 4, Problem 5, Bonus Problem external assistance, including ChatGPT, was utilized. Apart from this, ChatGPT was only used for language and formatting throughout the assignment. No other part of the content was generated by AI.

Problem 1

(a)

```
87(B)
```

```
  87(B)
  /
75(R)
```

```
    75(B)
   /    \
62(R)   87(R)
```

```
      75(B)
     /    \
  62(B)   87(B)
  /
20(R)
```

```
        75(B)
       /    \
   38(B)    87(B)
   /  \
20(R) 62(R)
```

```
          75(B)
         /    \
     38(R)    87(B)
     /  \
  20(B) 62(B)
  /
9(R)
```

(b)

```
        75(B)
       /    \
   38(R)    87(B)
   /  \
20(B) 62(B)
```

```
      75(B)
     /    \
 38(B)    87(B)
    \
    62(R)
```

```
    75(B)
   /    \
62(B)   87(B)
```

```
75(B)
    \
    87(R)
```

```
87(B)
```

Problem 2

Basic Idea

Use two heaps, a max-heap containing the smaller $n/2$ items and a min-heap containing the bigger $n/2$ elements.

On **INSERT**, compare the element with the element on the top of each heap and insert it into the apoprate one, then **POP** and **PUSH** elements through the two heaps so that the two heaps contain equal amount of elements (or differ by one).

On **MEDIAN**, return the element on the heap whith more elements.

As heap operations cost $O(\log n)$ time and on each **INSERT** or **MEDIAN** consist of $O(1)$ heap operations, the cost of operations if $O(\log n)$.

Pseudocode

```
class FastMedian:
    maxheap  # smaller half
    minheap  # larger half

    INITIALIZE():
        maxheap = new MaxHeap()
        minheap = new MinHeap()

    INSERT(x):
        if maxheap.is_empty() or x <= maxheap.peek():
            maxheap.push(x)
        else:
            minheap.push(x)
        if maxheap.size > minheap.size + 1:
            minheap.push(maxheap.pop())
        elif minheap.size > maxheap.size:
            maxheap.push(minheap.pop())
```

```
    MEDIAN():
        if maxheap.is_empty():
            return null
        return maxheap.peek()
```

Problem 3

Basic Idea

Use a tree like stucture, for each node, store index start and index end, indicating the start and end of the region the node refers to, the number of the first avalible room and the count of reaining avaliable rooms. The node's left and right child refers to two more ranges of half the size of the parent.

On **INITIALIZE**, the tree is built, as $2n = O(n)$ nodes are built, the time cost is $O(n)$.

On **COUNT**, recursively go into region of tree nodes that overlaps with $[l, h]$, as only two nodes in a layer may recursively call the next layer, and other nodes either return 0 or count, $O(\log n)$ nodes are called.

On **CHECKIN**, recursively finde the leftmost avaliable room, mark it occupied and propogate from that leaf to root, updating the count and first avalible along the way.

On **CHECKOUT**, mark the room avaliable and propogate from that leaf to root, updating the count and first avaliable along the way.

Pseudocode

```
class Node:
    start
    end
    count # number of available rooms
    first # first available room
    left
    right
    parent

class Hotel:
    available[]
    root
    INITIALIZE(n):
        available = array of size (n+1) initialized to all 1
        available[0] = 0
        BuildTree(l, r, parent):
            node = new Node()
            node.parent = parent
            node.start = l
            node.end = r
            if l == r:
                node.count = 1
                node.first = l
                node.left = null
                node.right = null
                return node
            mid = (l + r) / 2
            node.left = BuildTree(l, mid, node)
```

```
        node.right = BuildTree(mid + 1, r, node)
        node.count = node.left.count + node.right.count
        node.first = node.left.first
        return node

    root = BuildTree(1, n, null)

COUNT(l, h):
    CountDFS(l, r, node):
        if node.end < l or node.start > r:
            return 0
        if l <= node.start and node.end <= r:
            return node.count
        return CountDFS(l, r, node.left) + CountDFS(l, r, node.right)

    return CountDFS(l, h, root)

CHECKIN(l, h):
    FindFirst(l, r, node):
        if node.end < l or node.start > r:
            return null
        if node.count == 0:
            return null
        if node.start == node.end:
            if available[node.start] == 1 and l <= node.start <= r:
                return node.start
            return null
        leftResult = FindFirst(l, r, node.left)
        if leftResult != null:
            return leftResult
        return FindFirst(l, r, node.right)
    room = FindFirst(l, h, root)
    if room == null:
        return null
    available[room] = 0
    UpdateAfterCheckin(room, root)
    return room

UpdateAfterCheckin(room, node):
    if node == null:
        return
    if room < node.start or room > node.end:
        return
    if node.start == node.end:
        node.count = 0
        node.first = null
        return
    UpdateAfterCheckin(room, node.left)
    UpdateAfterCheckin(room, node.right)
    node.count = node.left.count + node.right.count
    if node.left.first != null:
        node.first = node.left.first
    else:
        node.first = node.right.first

CHECKOUT(x):
```

```
        available[x] = 1
        UpdateAfterCheckout(x, root)

    UpdateAfterCheckout(room, node):
        if room < node.start or room > node.end:
            return
        if node.start == node.end:
            node.count = 1
            node.first = room
            return
        UpdateAfterCheckout(room, node.left)
        UpdateAfterCheckout(room, node.right)
        node.count = node.left.count + node.right.count
        if node.left.first != null:
            node.first = node.left.first
        else:
            node.first = node.right.first
```

Problem 4

(a)

$$\begin{aligned} h(61) &= 700 \\ h(62) &= 318 \\ h(63) &= 936 \\ h(64) &= 554 \\ h(65) &= 172 \end{aligned}$$

(b)

Consider a string $s = a_w a_{w-1} \cdots a_1 a_0$.
The hash function is:

$$h(s) = \left(a_0(2^p)^0 + a_1(2^p)^1 + \cdots + a_w(2^p)^w\right) \bmod (2^p - 1)$$

For any character value a and position i, we have:

$$a \cdot (2^p)^i \bmod (2^p - 1) = a \cdot \underbrace{(2^p - 1 + 1)^i}_{\equiv 1^i \pmod{2^p - 1}} \bmod (2^p - 1) = a \bmod (2^p - 1)$$

This is because $2^p \equiv 1 \pmod{2^p - 1}$, so $(2^p)^i \equiv 1 \pmod{2^p - 1}$ for all $i \geq 0$.

$$h(s) = (a_0 + a_1 + a_2 + \cdots + a_w) \bmod (2^p - 1)$$

Therefore, the hash value only depends on the sum of character values, not their positions.

Problem 5

(a)

$$Q_k = \binom{n}{k}\left(\frac{1}{n}\right)^k\left(\frac{n-1}{n}\right)^{n-k}$$

using Stirling's approximation:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \le \frac{n^k}{k!} \le \left(\frac{en}{k}\right)^k$$

so

$$Q_k \le \left(\frac{en}{k}\right)^k \left(\frac{1}{n}\right)^k \left(\frac{n-1}{n}\right)^{n-k} \le \left(\frac{e}{k}\right)^k$$

(b)

$$\{M = k\} = \bigcup_{i=1}^{n} \{\text{exactly } k \text{ keys hash to slot } i\}$$

so

$$P_k = \Pr\left(\bigcup_{i=1}^{n} \{\text{exactly } k \text{ keys hash to slot } i\}\right) \le \sum_{i=1}^{n} Q_k = nQ_k$$

(c)

From (a) and (b), we have $n(e/k)^k < 1/n^2$.

$$\Rightarrow \quad k(\lg k - \lg e) > 3 \lg n.$$

to proof that a constant c exist for the claim in (c), we proof a constant exist for:

$$k(\lg k - \lg e) > \frac{c \lg n}{\lg \lg n}\left(\frac{\lg(c \lg n)}{\lg \lg \lg n} - \lg e\right) > 3 \lg n.$$

$$\left(\frac{c(\lg n) \lg(c \lg n)}{\lg \lg n \, 3 \lg n}\right) - \left(\frac{c \lg n \ \lg \lg \lg n}{3 \lg n \ \lg \lg n}\right) - \left(\frac{c \ \lg e \ \lg n}{3 \lg n \ \lg \lg n}\right) > 1.$$

When n is any constant, it is easy to find c to make this hold, when n is large:

$$\lim_{n \to \infty} \left(\frac{c(\lg n) \lg(c \lg n)}{\lg \lg n \, 3 \lg n} - \frac{c \lg n \ \lg \lg \lg n}{3 \lg n \ \lg \lg n} - \frac{c \ \lg e \ \lg n}{3 \lg n \ \lg \lg n}\right) = \lim_{n \to \infty} \frac{c \lg n \ \lg \lg n}{3 \lg n \ \lg \lg n} = \frac{c}{3}.$$

The limit is a constant number including c, thus a constant c exist to make the conclusion in (c) holds.

(d)

$$\mathbb{E}[M] = \sum_{i=1}^{n} iP_i$$

Let $k_0 = \dfrac{c \ \lg n}{\lg \lg n}$ when $k \ge k_0$, $k \ge \dfrac{c \ \lg n}{\lg \lg n}$.

$$\mathbb{E}[M] = \sum_{i=1}^{k_0} iP_i + \sum_{i=k_0+1}^{n} iP_i \le \sum_{i=1}^{k_0} 1 + \sum_{i=k_0+1}^{n} \frac{1}{n^2} n \le k_0 + \sum_{i=1}^{n} \frac{1}{n^2} = k_0 + 1 = O(k_0) = O\left(\frac{\lg n}{\lg \lg n}\right).$$

Bonus Problem

Priority $K \sim \mathrm{Geom}(1/2)$, $\mathbb{E}(K_{\max}) = \Theta(\log n)$.

number of nodes whose priority is k:

$$\mathbb{E}[\#k] = n \, 2^{k-1}$$

Now proof that on a single route from root to leaf, the expectation of number of nodes whose prority is k is constant.

Let $A_t = \mathbf{1}\{K_t \geq k\}$ and $B_t = \mathbf{1}\{K_t \geq k+1\}$.

then due to $K \sim \mathrm{Geom}(1/2)$

$$\Pr(B_t = 1 \mid A_t = 1) = \Pr(K_t \geq k+1 \mid K_t \geq k) = \tfrac{1}{2}$$

the number of k we get before getting a $k+1$ has expectation less than $1 = O(1)$ thus the expectation of numbers of nodes whose prority is k is constant.

Totally

$$\mathbb{E}[\text{depth of tree}] = \sum_{i=1}^{c\log n} \mathbb{E}[\text{number on path which priority } = i] = \sum_{i=1}^{c\log n} O(1) = O(\log n).$$